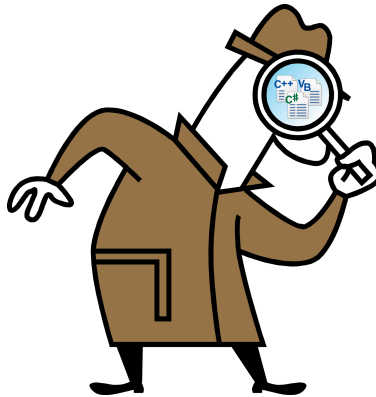


Llenguatges de Programació

Inferència de tipus



Gerard Escudero, Jordi Petit, Albert Rubio

UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Facultat d'Informàtica de Barcelona



Contingut

- Introducció
- Tipus i funcions prefefinides
- Funcions i condicionals
- Funcions d'ordre superior
- Patrons
- Classes
- Errors de tipus
- Exercicis

Inferència de tipus

La **inferència de tipus** és la detecció automàtica dels tipus de les expressions en un llenguatge de programació.

Permet fer més fàcils moltes tasques de programació, sense comprometre la seguretat de la comprovació de tipus.

Té sentit en llenguatges fortament tipats.

És una característica habitual dels llenguatges funcionals.

Alguns LPs amb inferència de tipus:

- C++ >= 11
- Haskell
- C#
- D
- Go
- Java >= 10
- Scala
- ...

Inferència de tipus a C++

La inferència de tipus apareix a la versió 11 de l'estàndard de C++.

- `auto`: Dedueix el tipus d'una variable a través de la seva inicialització:

```
map<int, string> m;  
auto x = 12;           // x és un int  
auto it = m.find(x);   // it és un map<int, string>::iterator
```

- `decltype`: Obté el tipus d'una expressió.

```
int x = 12;  
decltype(x + 1) y = 0;   // y és un int
```

Inferència de tipus a Haskell

- En la majoria de casos no cal definir els tipus.
- Es poden demanar els tipus inferits (que inclouen classes, si cal).

```
λ> :type 3 * 4
👉 3 * 4 :: Num a => a

λ> :type odd (3 * 4)
👉 odd (3 * 4) :: Bool
```

- Algunes situacions estranyes.
 - *Monomorphism restriction*: Sovint no es pot sobrecarregar una funció si no es dona una declaració explícita de tipus.

Inferència de tipus

Problema: Donat un programa, trobar el tipus més general de les seves expressions dins del sistema de tipus del LP.

Algorisme de Milner

- Curry i Hindley havien desenvolupat idees similars independentment en el context del λ -càlcul.
- Hindley–Milner i Damas–Milner
- L'algorisme és similar a la "unificació".



Foto: Domini públic

Propietats

- Complet.
- Computa el tipus més general possible sense necessitat d'anotacions.
- Eficient: gairebé lineal (inversa de la funció d'Ackermann). L'eficiència depèn de l'algorisme d'unificació que s'apliqui.

Algorisme de Milner

Descripció general

1. Es genera l'arbre de sintaxi de l'expressió (currificant totes les aplicacions).
2. S'etiqueta cada node de l'arbre amb un tipus:
 - Si el tipus és conegut, s'etiqueta amb aquest tipus.
 - Altrament, s'etiqueta amb una nova variable de tipus.
3. Es genera un conjunt de restriccions (d'igualtat principalment) a partir de l'arbre de l'expressió i les operacions que hi intervenen:
 - Aplicació,
 - Abstracció,
 - `let`, `where`,
 - `case`, guardes, patrons,
 - ...
4. Es resolen les restriccions mitjançant unificació.

Contingut

- Introducció
- Tipus i funcions prefefinides
- Funcions i conditionals
- Funcions d'ordre superior
- Patrons
- Classes
- Errors
- Exercicis

Haskell i λ -càlcul

Les expressions simples en **Haskell** es tradueixen a un **λ -càlcul** ampliat amb tipus simples i les seves funcions predefinides associades*:

- `Int`, `Bool`, `Char`...
- `3`, `True`...
- `+`, `-`, `*`, `==`, `&&`...

Exemples:

```
2 + 3           →  (+) 2 3
(+) 2 3         →  (+) 2 3
not True        →  not True
(\x -> x + x) 3  →  (\x.(+) x x) 3
```

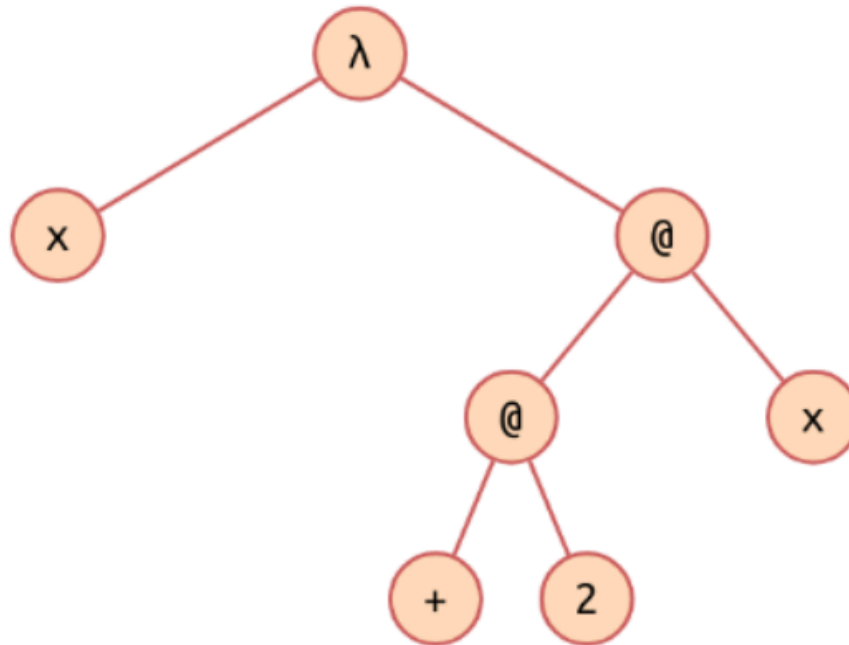
* **Model matemàtic:** λ -càlcul amb tipus ($\lambda \rightarrow$) (*simply typed lambda calculus*)

Primer exemple

Expressió currificada i sense operadors infixes:

```
\x -> 2 + x    →    \x -> (+) 2 x
```

Arbre de l'expressió currificada:

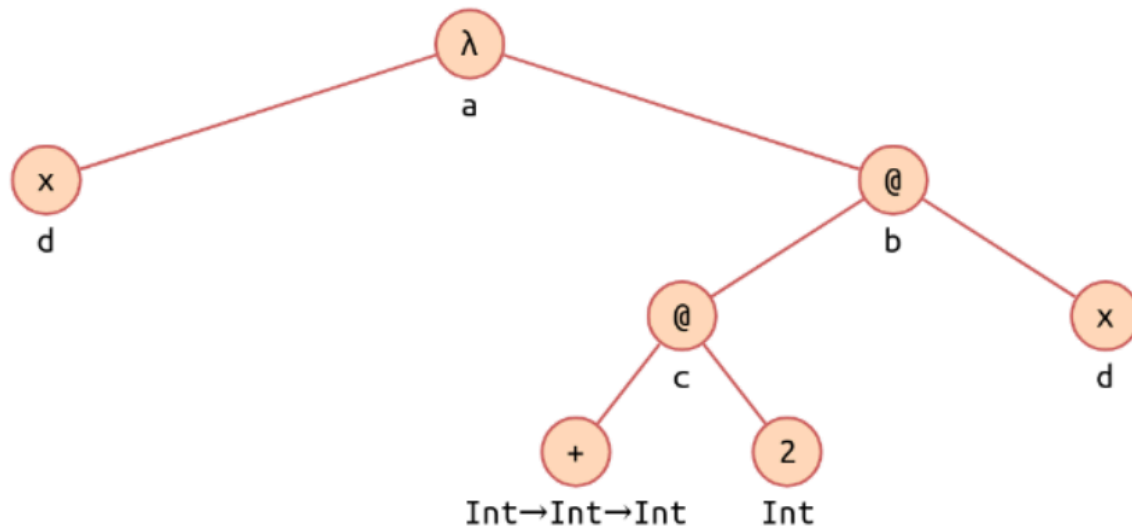


Primer exemple

```
\x -> (+) 2 x
```

Etiquetem els nodes:

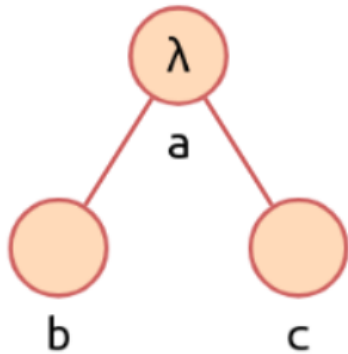
- Si el tipus és conegut, se'ls etiqueta amb el seu tipus.
- Altrament, se'ls etiqueta amb una nova variable de tipus.
- Nodes iguals han de tenir etiquetes iguals.



Algorisme de Milner

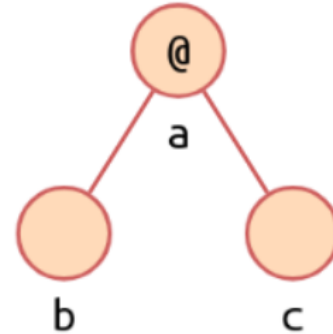
Regles per generar les equacions

- Abstracció:



- Equació: $a = b \rightarrow c$

- Aplicació:



- Equació: $b = c \rightarrow a$

Primer exemple

- Obtenim les equacions:

$a = d \rightarrow b$

$c = d \rightarrow b$

$\text{Int} \rightarrow \text{Int} \rightarrow \text{Int} = \text{Int} \rightarrow c$

- Solucionem les equacions:

$a = \text{Int} \rightarrow \text{Int}$

$b = \text{Int}$

$c = \text{Int} \rightarrow \text{Int}$

$d = \text{Int}$

- El tipus de l'expressió és el de l'arrel (a):

$\backslash x \rightarrow (+) 2 x :: \text{Int} \rightarrow \text{Int}$

- Recordeu: $\rightarrow$ associa per la dreta: $a \rightarrow b \rightarrow c = a \rightarrow (b \rightarrow c)$
- Recordeu: aplicació associa per l'esquerra: $f x y = (f x) y$

Àmbits locals

Traducció:

```
let v = B in E  ≡  (λv.E) B
```

Exemple:

Haskell:

```
let x = 1  
    y = 2  
in (x + y)
```

λ-càlcul:

```
(λx.λy.(+) x y) 1 2
```

where:

Es codifica de forma similar.

Contingut

- Introducció
- Tipus i funcions predefinits
- Funcions i condicionals
- Funcions d'ordre superior
- Patrons
- Classes
- Errors
- Exercicis

Funcions

Supercombinador: funció que només conté referències als seus paràmetres o a altres *supercombinadors*.

Exemples:

```
inc x = x + 1  
mes2 x = inc (inc x)
```

Traducció:

```
f x y = E  ≡  λx.λy.E
```

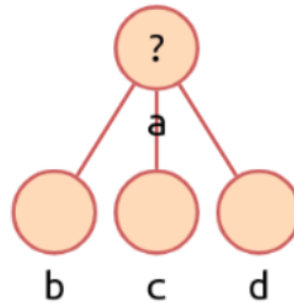
Exercici

```
inc :: Int -> Int  
:t mes2
```

Condicionals

S'exten el λ -càlcul amb un "if lazy".

- Condicional if-then-else:



- Equacions:
 - $b = \text{Bool}$
 - $a = c = d$
- Les guardes es codifiquen amb aquesta estructura.

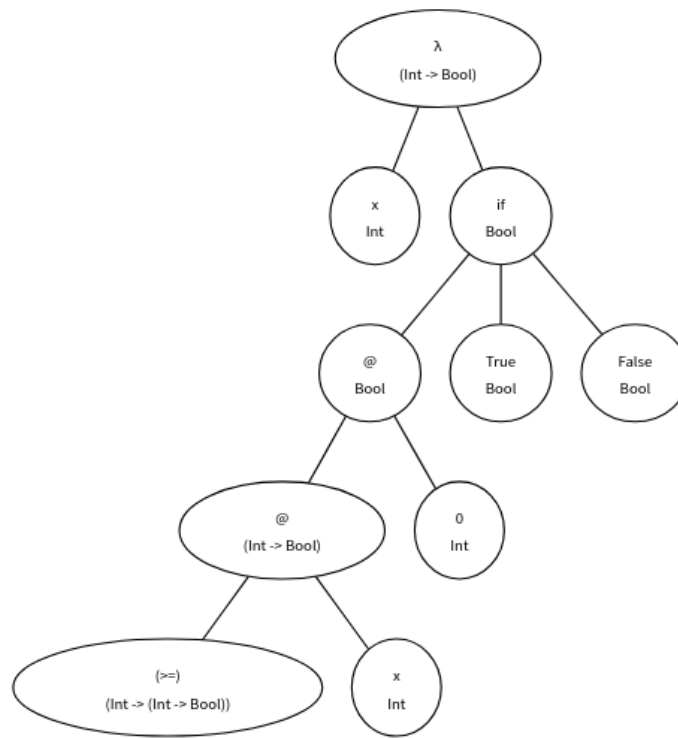
Exercici

```
positiu x
| x >= 0 = True
| otherwise = False
```

Exercici

```
positiu x
| x >= 0 = True
| otherwise = False
```

Inferència de tipus:



Contingut

- Introducció
- Tipus i funcions predefinits
- Funcions i condicionals
- Funcions d'ordre superior
- Patrons
- Classes
- Errors
- Exercicis

Composició

Exemple:

```
((.) ((+) 1) ((*) 2)) 3
```

Implementació:

```
(.) ≡ λf.λg.λx.f(gx)
```

S'ha d'afegir **polimorfisme** a la inferència de tipus (variables de tipus):

```
(.) :: (b -> c) -> (a -> b) -> a -> c
```

Exercici

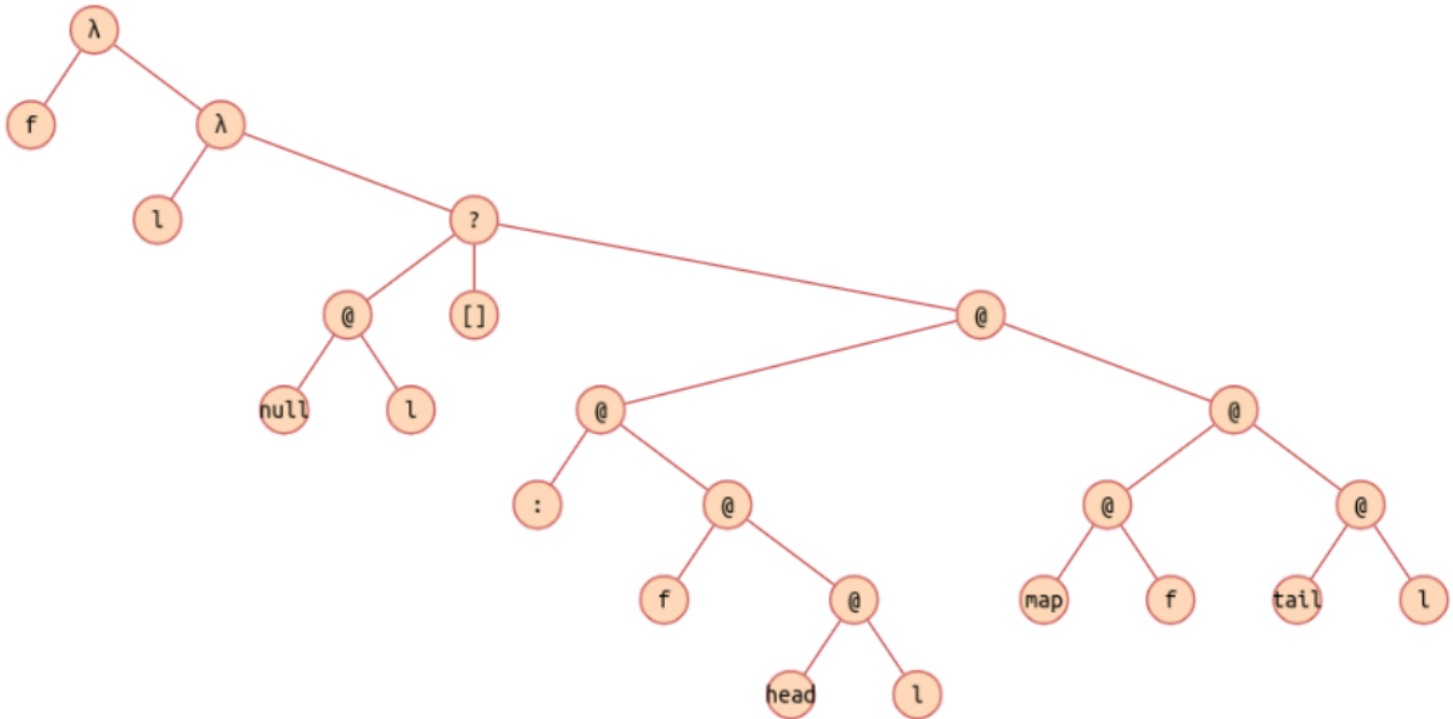
```
mes1 :: Int -> Int  
per2 :: Int -> Int  
:t mes1 . per2
```

* **Model matemàtic**: λ-càlcul polimòrfic (System F o λ-càlcul de 2on ordre). Introdueix un mecanisme de quantificació universal (lògica amb $\forall x$) sobre els tipus.

Example

```
map f l = if null l then [] else f (head l) : map f (tail l)
```

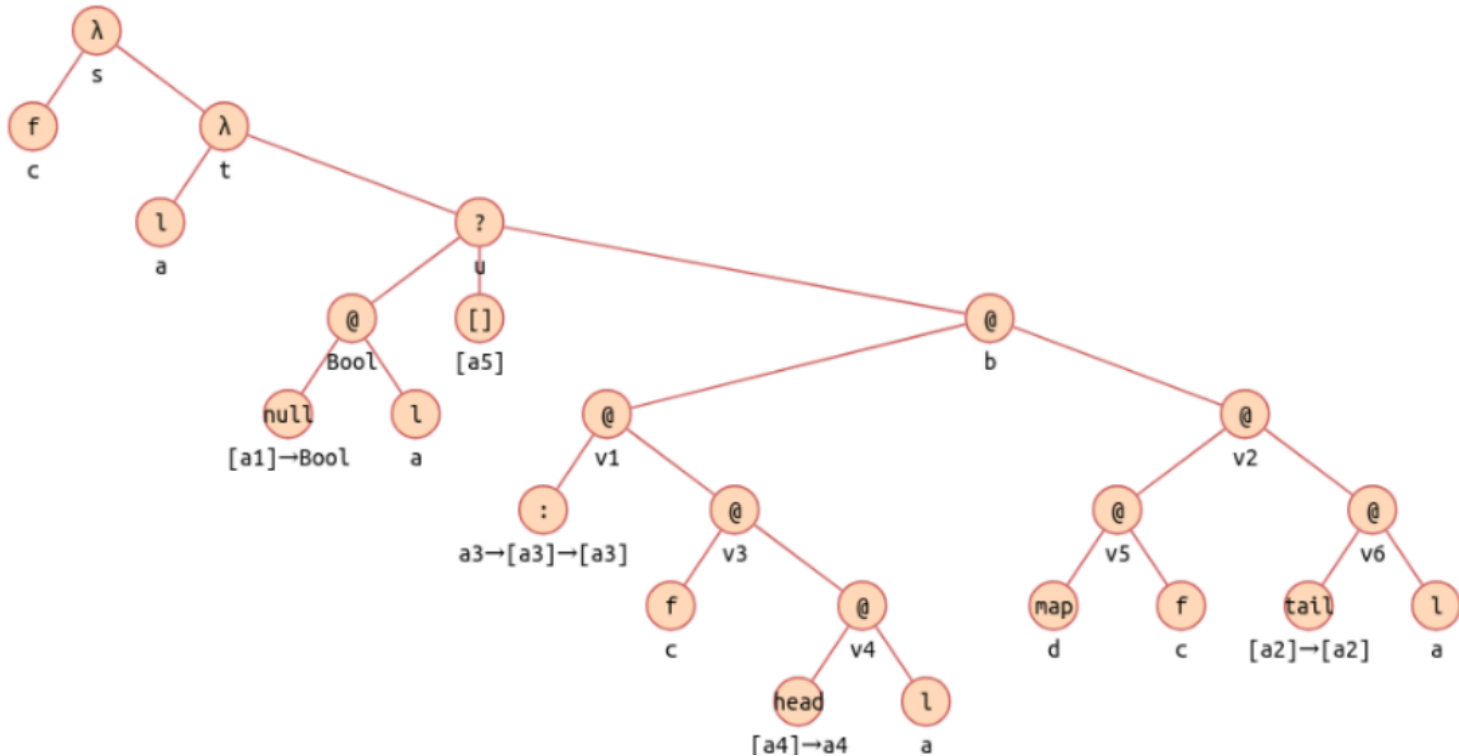
```
\f -> \l -> if null l then [] else f (head l) : map f (tail l)
```



Exemple

```
\f -> \l -> if null l then [] else f (head l) : map f (tail l)
```

Arbre etiquetat amb tipus:



Example

```
\f -> \l -> if null l then [] else f (head l) : map f (tail l)
```

Equacions:

- $s = c \rightarrow t$
- $t = a \rightarrow u$
- $u = [a5]$
- $u = b$
- $[a1] \rightarrow Bool = a \rightarrow Bool$
- $v1 = v2 \rightarrow b$
- $a3 \rightarrow [a3] \rightarrow [a3] = v3 \rightarrow v1$
- $c = v4 \rightarrow v3$
- $[a4] \rightarrow a4 = a \rightarrow [v4]$
- $v5 = v6 \rightarrow v2$
- $d = c \rightarrow v5$
- $[a2] \rightarrow [a2] = a \rightarrow v6$
- $s = d$ (per establir que el `map` té el mateix tipus a la definició i a l'ús recursiu)

Example

```
\f -> \l -> if null l then [] else f (head l) : map f (tail l)
```

Solució:

- `a = [a1]`
- `a2 = a1`
- `a4 = a1`
- `a5 = a3`
- `b = [a3]`
- `c = a1 → a3`
- `v1 = [a3] → [a3]`
- `v2 = [a3]`
- `v3 = a3`
- `v4 = a1`
- `v5 = [a1] → [a3]`
- `v6 = [a1]`
- `d = (a1 → a3) → [a1] → [a3]`
- `s = (a1 → a3) → [a1] → [a3] (arrel)`

Contingut

- Introducció
- Tipus i funcions predefinits
- Funcions i condicionals
- Funcions d'ordre superior
- Patrons
- Classes
- Errors
- Exercicis

Definició de funció amb patrons

```
map f (x : xs) = f x : map f xs
```

En aquest cas la introducció de lambdes és una mica diferent, ja que tractem els patrons com si fossin variables lliures:

```
\f -> \ (x : xs) -> f x : map f xs
```

Noteu que ara hem de considerar que el primer argument de la lambda pot ser una expressió, que tractarem igual que les demés.

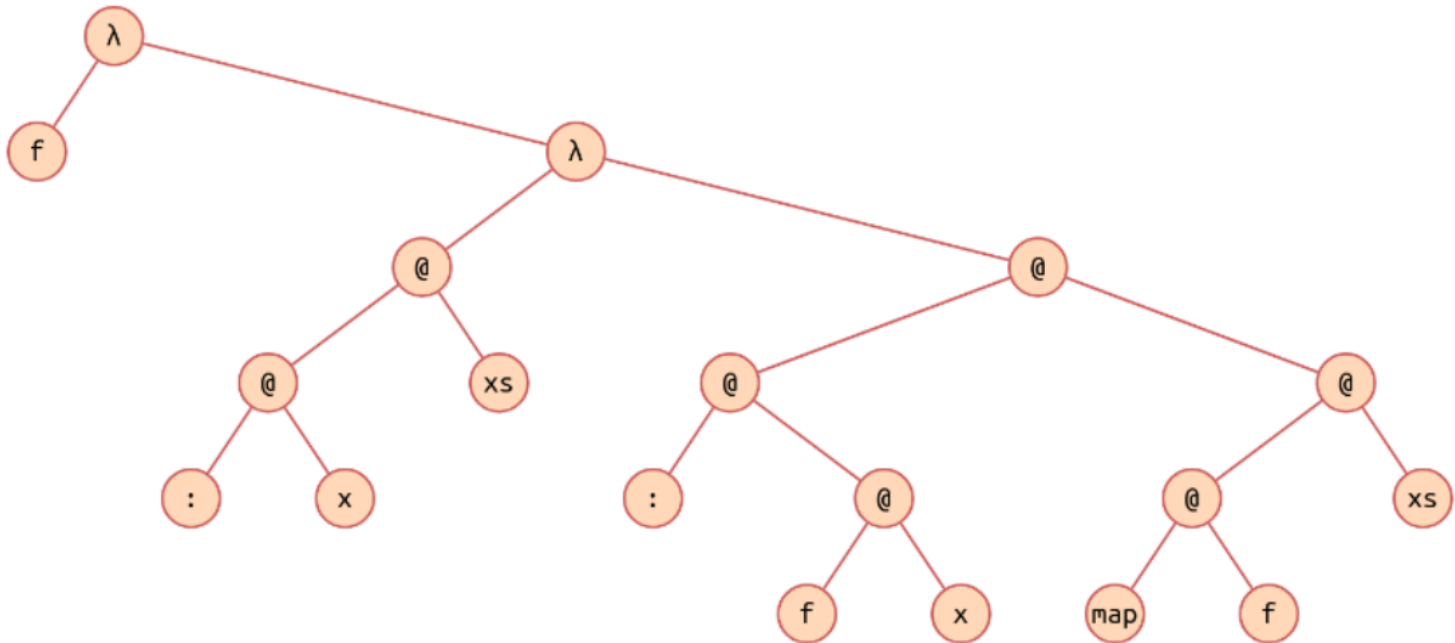
Totes les variables del patró queden lligades per la lambda.

En l'avaluació d'expressions amb patrons, el sistema ha de fer matching entre el paràmetre i el patró per assegurar-se de que l'aplicació és vàlida.

Algorisme de Milner

```
\f -> \ (x : xs) -> f x : map f xs
```

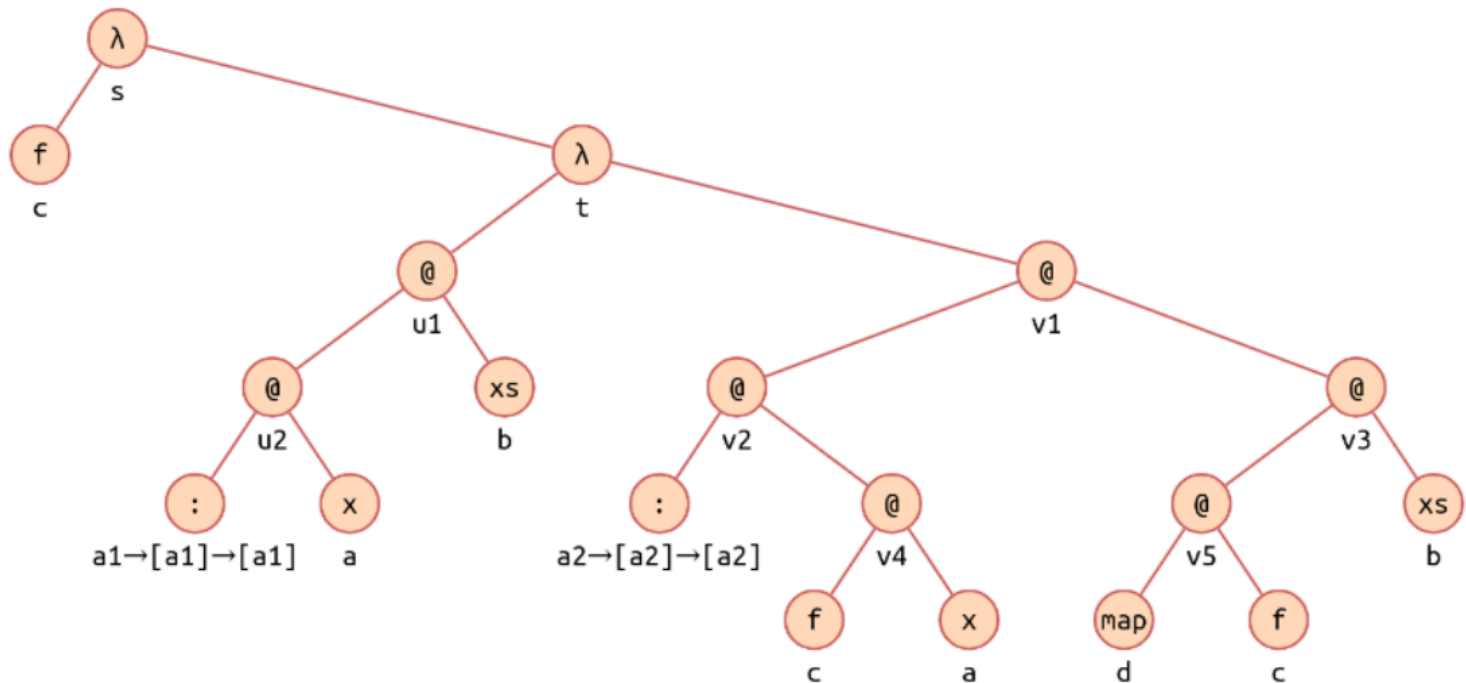
Arbre de l'expressió:



Algorisme de Milner

```
\f -> \ (x : xs) -> f x : map f xs
```

Arbre etiquetat amb tipus:



Algorithme de Milner

```
\f -> \ (x : xs) -> f x : map f xs
```

Equations:

- $s = c \rightarrow t$
- $t = u1 \rightarrow v1$
- $u2 = b \rightarrow u1$
- $a1 \rightarrow [a1] \rightarrow [a1] = a \rightarrow u2$
- $v2 = v3 \rightarrow v1$
- $a2 \rightarrow [a2] \rightarrow [a2] = v4 \rightarrow v2$
- $c = a \rightarrow v4$
- $v5 = b \rightarrow v3$
- $d = c \rightarrow v5$

- $s = d$

Algorisme de Milner

```
\f -> \ (x : xs) -> f x : map f xs
```

Solució:

- $a1 = a$
- $b = [a]$
- $c = a \rightarrow a2$
- $d = (a \rightarrow a2) \rightarrow [a] \rightarrow [a2]$
- $s = (a \rightarrow a2) \rightarrow [a] \rightarrow [a2]$
- $t = [a] \rightarrow [a2]$
- $u1 = [a]$
- $u2 = [a] \rightarrow [a]$
- $v1 = [a2]$
- $v2 = [a2] \rightarrow [a2]$
- $v3 = [a2]$
- $v4 = a2$
- $v5 = [a] \rightarrow [a2]$

Per tant, el tipus de l'arrel és $s = (a \rightarrow a2) \rightarrow [a] \rightarrow [a2]$.

Funcions amb més d'una definició

```
map f [] = []  
map f (x : xs) = f x : map f xs
```

Quan hi ha més d'una definició, apareix un bosc d'arbres.

Les definicions per la mateixa funció tenen el mateix tipus a l'arrel*.

Nota: analitzant una sola definició, el tipus pot ser més general que l'esperat:

```
foldr f z (x : xs) = f x (foldr f z xs)
```

```
foldr :: (t1 -> t2 -> t2) -> t3 -> [t1] -> t2 ⚠
```

```
foldr f z (x : xs) = f x (foldr f z xs)  
foldr f z [] = z
```

```
foldr :: (t1 -> t2 -> t2) -> t2 -> [t1] -> t2 🔥
```

* Per tractar patrons múltiples s'exten el λ -càlcul amb l'operador λ , que agrupa les diferents definicions d'una mateixa funció en una única expressió.

Contingut

- Introducció
- Tipus i funcions predefinits
- Funcions i condicionals
- Funcions d'ordre superior
- Patrons
- Clases
- Errors
- Exercicis

Classes

La presència de definicions com ara

```
(+) :: Num a => a -> a -> a  
(>) :: Ord a => a -> a -> Bool
```

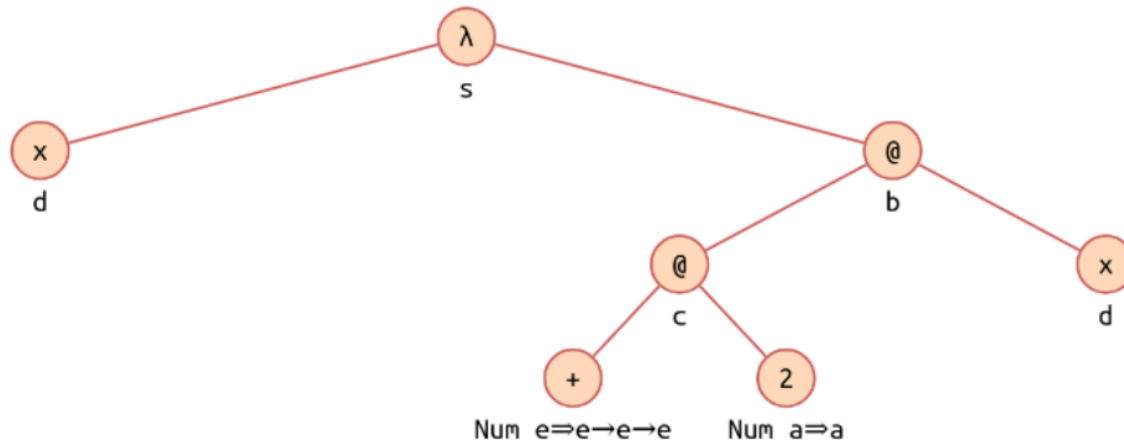
introdueix unes noves *restriccions de context*.

Per tant, les solucions també han de contenir i satisfer les condicions de classe.

Classes

f $x = 2 + x$

Arbre etiquetat:



Classes

Equacions:

- $s = d \rightarrow b$
- $c = d \rightarrow b$
- $e \rightarrow e \rightarrow e = a \rightarrow c$

Restriccions:

- $\text{Num } a$
- $\text{Num } e$

Solució:

- $s = a \rightarrow a$
- $b = a$
- $c = a \rightarrow a$
- $d = a$
- $e = a$

El tipus de l'arrel (de f) és doncs $\text{Num } a \Rightarrow a \rightarrow a$.

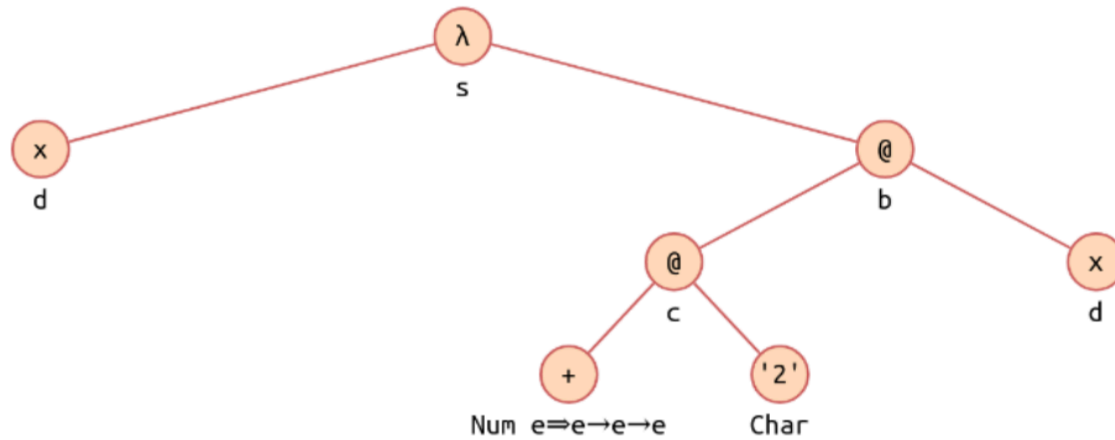
Contingut

- Introducció
- Tipus i funcions predefinits
- Funcions i condicionals
- Funcions d'ordre superior
- Patrons
- Classes
- Errors
- Exercicis

Errors

f $x = '2' + x$

Arbre etiquetat:



Errors

Equacions:

- $s = d \rightarrow b$
- $c = d \rightarrow b$
- $e \rightarrow e \rightarrow e = \text{Char} \rightarrow c$

Restriccions:

- $\text{Num } e$

Intent de solució:

- $s = \text{Char} \rightarrow \text{Char}$
- $b = \text{Char}$
- $c = \text{Char} \rightarrow \text{Char}$
- $d = \text{Char}$
- $e = \text{Char}$
- Num Char ❌

Perquè Char no és instància de Num !

Contingut

- Introducció
- Tipus i funcions predefinits
- Funcions i condicionals
- Funcions d'ordre superior
- Patrons
- Classes
- Errors
- Exercicis

Exercicis

- Utilitzeu l'algorisme de Milner per inferir el tipus de:

```
2 + 3 + 4
```

- Utilitzeu l'algorisme de Milner per inferir el tipus de:

```
2 + 3 <= 2 + 2
```

- Utilitzeu l'algorisme de Milner per inferir el tipus de:

```
map (* 2)
```

(Suposeu `(*) :: Int -> Int -> Int`)

- Utilitzeu l'algorisme de Milner per inferir el tipus de:

```
foldl (flip (:)) []
```

Exercicis

- Utilitzeu l'algorisme de Milner per inferir el tipus de:

```
\f x -> f $ f x
```

- Utilitzeu l'algorisme de Milner per inferir el tipus de:

```
\f -> f . f
```

- Utilitzeu l'algorisme de Milner per inferir el tipus de:

```
\x y -> if y /= 0 then Just (x `div` y) else Nothing
```

(Suposeu `div :: Int -> Int -> Int`)

- Utilitzeu l'algorisme de Milner per inferir el tipus de:

```
\xs ys -> zipWith (,) xs ys
```

Exercicis

- Inferiu el tipus de:

```
ones = 1 : ones
```

- Inferiu el tipus de:

```
even x = if rem x 2 == 0 then True else False
```

amb `rem :: Int → Int → Int`.

- Inferiu el tipus de:

```
even x = rem x 2 == 0
```

- Inferiu el tipus de:

```
last [x] = x
```

Recordeu que `[x]` és `x:[]`.

Exercicis

- Inferiu el tipus de:

```
delete x (y:ys) =  
  if x == y  
  then ys  
  else y : delete x ys
```

`amb (==) :: Eq a ⇒ a → a → Bool.`